Chapter 7: Design Rules

7.1 Introduction

Design Rules: Rules that **help designers make systems more usable.**

Design rules can be classified along two dimensions

1. **Authority:** Whether a rule **must be followed** or is only a suggestion.
2. **Generality:** Whether a rule applies to many **situations** or **only specific cases.**

> Types of Design Rules

Type of rule	Authority	Generality	Character
Principles	Low	High	Broad, abstract rules
Standards	High	Low (limited)	Specific design rules; **must be followed; mandatory rules**
Guidelines	Lower	medium	**Recommended** practices

Why Use Design Rules?

- Reduce poor design choices.
- Improve usability.
- Most useful in **early design stages.**
- Rules may conflict, requiring trade-offs.

7.2 Principles to Support Usability

1. Learnability: How easily new users learn the system.

2. Flexibility: How many ways users and systems can interact.

3. Robustness: How well the system supports users in achieving goals.

7.2.1 Learnability - PSGCF

Learnability concerns the features that let novice users understand how to use a system initially and then attain maximal performance. Its supporting principles are summarised below.

Principle	Definition	Related principles
Predictability	Ability to predict the result of future actions. Knowing what will happen after a button is clicked Operation visibility: • Users can easily see available actions. • Supports recognition instead of recall.	**Operation visibility**
Synthesizability	assessing **effect of past operations** on the current state System clearly shows state changes. Changes shown instantly.User must check manually. • Immediate: instant feedback • Eventual: delayed but accurate feedback	Immediate / eventual honesty
Familiarity	extent to which a user's real-world or other computer knowledge can be applied to a new system **Guessability:** User can guess how the system works. **Affordance:** Appearance suggests usage.	Guessability, affordance

Generalizability	Applying learned knowledge to new situations.	–
Consistency	Likeness in input–output behaviour arising from similar situations or task objectives → heart icon means like Similar situations behave similarly. Similar inputs produce similar effects.	–

7.2.2 Flexibility - DTSMC

Flexibility refers to the **multiplicity of ways the user and system exchange information.**

Principle	Definition	Related principles
Dialog initiative	Who controls the interaction? **System pre-emptive**: System controls interaction. **User pre-emptive:** User controls interaction.	System / user pre-emptiveness
Multi-threading	Supporting multiple tasks • **Concurrent** → tasks run together • **Interleaving** → switching between tasks	Concurrent vs interleaving, multi-modality
Task migratability	Control can move between user and system. Eg; Spell checker checks spelling while user decides corrections.	–
Substitutivity	Different ways to perform the same task. **Input Substitutivity:** Multiple input methods. **Representation Multiplicity:** Same information shown differently. **Equal Opportunity:** Output can also become input.	Representation multiplicity, equal opportunity

Customizability	Users or system can modify interface • **Adaptability:** User changes interface. • **Adaptivity:** System changes automatically.	Adaptivity, adaptability

7.2.3 Robustness - TORR

Robustness covers **features that support successful achievement and assessment of goals.**

Principle	Definition	Related principles
Observability	Ability to **understand the system state** **Browsability:** Explore information safely. **Defaults:** Suggested values. **Static Defaults:** Remain unchanged. **Dynamic Defaults:** Change based on usage. **Reachability:** Ability to move between states. **Persistence:** Information remains visible.	Browsability, static/dynamic defaults, reachability, persistence, operation visibility
Recoverability	**Ability to recover from errors.** **Forward Recovery:** Continue from **current state.** **Backward Recovery:** Return to previous state (**Undo**). **Commensurate Effort:** **Hard-to-reverse actions** should **require more effort**	forward/backward recovery, commensurate effort
Responsiveness	**Speed of system response.** Similar actions should take similar time.	Stability

Task conformance	**System supports user tasks correctly.** **Task Completeness:** Supports all required tasks. **Task Adequacy:** Supports tasks the way users understand them.	Task completeness, task adequacy

7.3 Standards

Mandatory design rules established by organizations.

> Characteristics

Hardware Standards

- Based on **ergonomics**[designing and arranging environments, products, and systems to fit the people who use them.] and **physiology**.
- More **precise and stable.**

Software Standards

- Based on **psychology** and **cognition**.
- More **general and flexible**

> hardware is harder and more expensive to change than (flexible) software, so hardware requirements change less often. Hw standards r more common than sw

> ISO Definition of Usability

Usability — the **effectiveness, efficiency** and **satisfaction** with which specified users achieve specified goals.

- **Effectiveness** → Achieving goals accurately.
- **Efficiency** → Resources used to achieve goals.
- **Satisfaction** → User comfort and acceptance.

Authority in Practice

- A standard is powerful only when many people actually use it.
- Publishing a standard does **not** automatically make it mandatory.
- Many software standards are guidelines rather than strict rules.
- Some technologies become **de facto standards** through widespread use before any official standard exists (e.g., the **X Window System**).
- Standards were first common in **safety-critical systems** (aircraft, nuclear plants) because design errors can be very costly.
- Usability standards became important later when organizations realized that poorly designed software causes user frustration, errors, and productivity loss.

7.4 Guidelines

Recommended design practices.

Characteristics

- Less strict than standards.
- Easier to apply.
- Often **technology-specific**
- suggestive.

Smith & Mosier Guidelines

Six categories:

- Data Entry
- Data Display
- Sequence Control
- User Guidance
- Data Transmission
- Data Protection

Dialog styles

A major concern across general guidelines is **dialog styles** — the means by which the user communicates input and how the system presents the communication device. Smith and Mosier list eight; Mayhew lists seven (the only real difference is query languages, which can be treated as a special case of command languages).

Smith & Mosier (8)	Mayhew (7)
Question and answer	Question and answer
Form filling	Fill-in forms
Menu selection	Menus
Function keys (shortcuts)	Function keys
Command language	Command language
Query language	–
Natural language	Natural language
Graphic selection	Direct manipulation

Platform-specific guidelines and style guides

Style guides → Provide platform-specific recommendations.

7.5 Golden Rules and Heuristics

- Simple usability rules that summarize good design practices.

7.5.1 Shneiderman's Eight Golden Rules

CSFCEPUM (**C**lever **S**tudents **F**inish **C**lasses **E**arly, **P**assing **U**niversity **M**odules)

1. Strive for Consistency: Keep layouts, terminology, and actions consistent / make things similar

2. Enable Shortcuts: Support expert users with faster methods.

3. Offer Informative Feedback: Provide feedback for every action.

4. Design Dialogs for Closure: Clearly indicate task completion.

5. Prevent Errors: Avoid errors and provide recovery help.

6. Permit Easy Reversal: Support Undo operations.

7. Support User Control: Users should feel in charge.

8. Reduce Memory Load: Minimize information users must remember.

7.5.2 Norman's Seven Principles

KSVMCES - Karma Says Very Mean Cats Eat Sushi

1. Use Knowledge in the World and in the Head: Provide information while allowing learning / Balance visible cues on screen (world) with user memory (head)

2. Simplify Tasks: Reduce complexity and memory demands / break complex tasks into smaller steps

3. Make Things Visible: Show possible actions and results clearly.

4. Get the Mappings Right: Controls should match user expectations.

5. Use Constraints: Prevent incorrect actions.

6. Design for Error: Expect mistakes and support recovery.

7. Standardize: Use common conventions when possible.

7.6 HCI Patterns

reusable solution to a recurring design problem.

> Characteristics

- Based on successful design practice.
- Describes what to do and why.
- Works at different levels of design.
- Easy to understand and communicate.

> Pattern Language

A **collection of connected patterns** that **help create complete designs.**

Example

Problem: Users get lost in websites.

Pattern: Provide a safe place to return (Home button, breadcrumbs).

Benefit: Helps users recover and continue navigation confidently.